

# 西安电子科技大学

## 计算机组织与体系结构课程设计 课程实验报告

实验名称 基本模型机的设计与实现

计算机科学与技术 学院 2303013 班

姓名 郑墨涵 学号 23009201247

同作者 无

实验日期 2025 年 9 月 13、14 日

实验地点 E-II-522 实验批次

成 绩

指导教师评语：

指导教师：

\_\_\_\_年\_\_\_\_月\_\_\_\_日

### 实验报告内容基本要求及参考格式

- 一、实验目的
- 二、实验所用仪器（或实验环境）
- 三、实验基本原理及步骤（或方案设计及理论计算）
- 四、实验数据记录（或仿真及软件设计）
- 五、实验结果分析及回答问题（或测试环境及测试结果）

# 计算机组织与体系结构课程设计

## (一) 基本模型机设计与实现

### 一. 实验目的

1. 深入理解基本模型计算机的功能、组成知识;
2. 深入学习计算机各类典型指令的执行流程;
3. 学习微程序控制器的设计过程和相关技术, 掌握 LPM\_ROM 的配置方法。
4. 在掌握部件单元电路实验的基础上, 进一步将单元电路组成系统, 构造一台基本模型计算机。
5. 定义五条机器指令, 并编写相应的微程序, 上机调试, 掌握计算机整机概念。掌握微程序的设计方法, 学会编写二进制微指令代码表。
6. 通过熟悉较完整的计算机的设计, 全面了解并掌握微程序控制方式计算机的设计方法。

### 二. 实验原理

1. 在部件实验过程中, 各部件单元的控制信号是人为模拟产生的, 而本实验将能在微过程控制下自动产生各部件单元控制信号, 实现特定的功能。实验中, 计算机数据通路的控制将由微过程控制器来完成, CPU 从内存中取出一条机器指令到指令执行结束的一个指令周期, 全部由微指令组成的序列来完成, 即一条机器指令对应一个微程序。

#### 2. 指令格式

##### (1) 指令格式

采用寄存器直接寻址方式, 其格式如下:

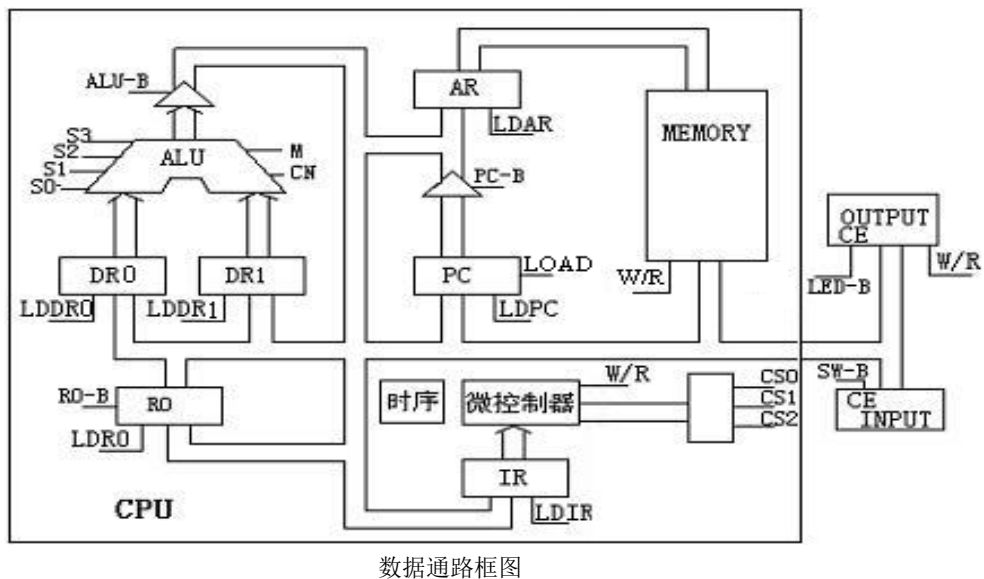
|    |         |   |   |   |    |   |    |   |
|----|---------|---|---|---|----|---|----|---|
| 位  | 7       | 6 | 5 | 4 | 3  | 2 | 1  | 0 |
| 功能 | OP-CODE |   |   |   | rs |   | rd |   |

本实验采用五条机器指令: IN(输入)、SUB(二进制减法)、AND(相与)、DEC(自减)、INV(取反)、SWAP(交换), 其指令格式如下(最高 4 位二进制数为操作码):

其中, OP-CODE 为操作码, rs 为源寄存器, rd 为目的寄存器, 并规定:

|  |  | Rs 或 rd | 选定的寄存器 |
|--|--|---------|--------|
|  |  | 00      | R0     |
|  |  | 01      | R1     |
|  |  | 10      | R2     |

| 助记符       | 机器指令码 | Addr 地址码 | 功能说明                                |
|-----------|-------|----------|-------------------------------------|
| IN        | 0 0H  |          | “INPUT” 中的数据→R0                     |
| SUB addr  | 1 0H  | 10 H     | DR1 - DR2 →R0                       |
| AND addr  | 2 0H  | 00 H     | DR1 & DR2→R0                        |
| DEC addr  | 3 0H  | 00 H     | DR1-1 → R0                          |
| INV addr  | 4 0H  | 11 H     | DR1*→R0                             |
| SWAP addr | 0 0H  | 00 H     | R0→RAM; DR2→R0;<br>R0→DR1; RAM→DR2; |



其中 IN 为单字长（8 位二进制），其余为双字长指令，XX H 为 addr 对应的十六进制地址码。为了向 RAM 中装入程序和数据，检查写入是否正确，并能启动程序执行，还必须设计三个控制台操作微程序。

- 1、存储器读操作（KRD）：下载实验程序后按总清除按键（CLR）后，控制台 SWA、SWB 为“00”时，可对 RAM 连续手动读出操作。
- 2、存储器写操作（KWE）：下载实验程序后按总清除按键（CLR）后，控制台 SWA、SWB 为“01”时，可对 RAM 连续手动写操作。
- 3、启动程序（RP）：下载实验程序后按总清除按键（CLR）后，控制台 SWA、SWB 为“11”时，即可转入到微地址“01”号“取指令”微指令，启动程序运行。

| SWB | SWA | 控制台指令    |
|-----|-----|----------|
| 0   | 0   | 读内存（KRD） |
| 0   | 1   | 写内存（KWE） |
| 1   | 1   | 启动程序（RP） |

24 位微代码定义：

|    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |   |     |   |     |     |     |     |     |   |
|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|---|-----|---|-----|-----|-----|-----|-----|---|
| 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8   | 7 | 6   | 5   | 4   | 3   | 2   | 1 |
| S3 | S2 | S1 | S0 | M  | Cn | WE | A9 | A8 | A  |    |    | B  |    | C  |   | uA5 |   | uA4 | uA3 | uA2 | uA1 | uA0 |   |

A、B、C 各字段功能说明：

| A 字段 |    |    |       | B 字段 |    |    |      | C 字段 |   |   |       |
|------|----|----|-------|------|----|----|------|------|---|---|-------|
| 15   | 14 | 13 | 选择    | 12   | 11 | 10 | 选择   | 9    | 8 | 7 | 选择    |
| 0    | 0  | 0  |       | 0    | 0  | 0  |      | 0    | 0 | 0 |       |
| 0    | 0  | 1  | LDRi  | 0    | 0  | 1  | RS-B | 0    | 0 | 1 | P (1) |
| 0    | 1  | 0  | LDDR1 | 0    | 1  | 0  |      | 0    | 1 | 0 |       |
| 0    | 1  | 1  | LDDR2 | 0    | 1  | 1  |      | 0    | 1 | 1 |       |

|   |   |   |      |   |   |   |       |   |   |   |       |
|---|---|---|------|---|---|---|-------|---|---|---|-------|
| 1 | 0 | 0 | LDIR | 1 | 0 | 0 |       | 1 | 0 | 0 | P (4) |
| 1 | 0 | 1 | LOAD | 1 | 0 | 1 | ALU-B | 1 | 0 | 1 | LDAR  |
| 1 | 1 | 0 | LDAR | 1 | 1 | 0 | PC-B  | 1 | 1 | 0 | LDPC  |

#### 24 位微代码中各信号的功能

- (1) uA5—uA0: 微程序控制器的微地址输出信号, 是下一条要执行的微指令的微地址。
- (2) S3、S2、S1、S0: 由微程序控制器输出的 ALU 操作选择信号, 以控制执行 16 种算术操作或 16 种逻辑操作中的某一种操作。
- (3) M: 微程序控制输出的 ALU 操作方式选择信号端。M=0 执行算术操作; M=1 执行逻辑操作。
- (4) /Cn: 微程序控制器输出的进位信号, /Cn=0 表示 ALU 运算时最低位有进位, /Cn=1 则表示无进位。
- (5) WE: 微程序控制器输出的 RAM 控制信号。当/CE=0 时, 如 WE=0 为存储器读; 如 WE=1 为存储器写。
- (6) A9、A8——译码后产生 CS0、CS1、CS2 信号, 分别作为 SW\_B、RAM、LED 的选通控制信号。
- (7) A 字段 (15、14、13) ——译码后产生与总线相连接的各单元的输入选通信号 (见表 6-1)。
- (8) B 字段 (12、11、10) ——译码后产生与总线相连接的各单元的输出选通信号。
- (9) C 字段 (9、8、7) ——译码后产生分支判断测试信号 P(1)~P(4)和 LDPC 信号。

系统涉及到的微程序流程见图 1-2。当执行“取指令”微指令时, 该微指令的判断测试字段为 P(1)测试。由于“取指令”微指令是所有微程序都使用的公用微指令, 因此 P(1)的测试结果出现多路分支。用指令寄存器的高 4 位 (IR7-IR4) 作为测试条件, 出现 5 路分支, 占用 5 个固定地址单元。

控制台操作为 P(4)测试, 它以控制台信号 SWB、SWA 作为测试条件, 出现了 3 路分支, 占用 3 个固定微地址单元。当分支微地址单元固定后, 剩下的其它地方就可以一条微指令占用控制存储器的一个微地址单元填写。

当全部微程序设计完毕后, 将每条微指令代码化转化成“二进制微代码”。

| 微地址 | 微指令    | 24 位微代码   |     |     |     |        |  |
|-----|--------|-----------|-----|-----|-----|--------|--|
| 00  | 018110 | 000000011 | 000 | 000 | 100 | 010000 |  |
| 01  | 01ED82 | 000000011 | 110 | 110 | 110 | 000010 |  |
| 02  | 00C048 | 000000001 | 100 | 000 | 001 | 001000 |  |
| 03  | 059A04 | 000001011 | 001 | 101 | 000 | 000100 |  |
| 04  | 01A205 | 000000011 | 010 | 001 | 000 | 000101 |  |
| 05  | 01E206 | 000000011 | 110 | 001 | 000 | 000110 |  |
| 06  | 001007 | 000000000 | 001 | 000 | 000 | 000111 |  |
| 07  | 03820E | 000000111 | 000 | 001 | 000 | 001110 |  |
| 10  | 001001 | 000000000 | 001 | 000 | 000 | 000001 |  |
| 11  | 01A203 | 000000011 | 010 | 001 | 000 | 000011 |  |
| 12  | 01E20F | 000000011 | 110 | 001 | 000 | 001111 |  |
| 13  | 01E218 | 000000011 | 110 | 001 | 000 | 011000 |  |
| 16  | 019A01 | 000000011 | 001 | 101 | 000 | 000001 |  |
| 17  | 00A015 | 000000001 | 010 | 000 | 000 | 010101 |  |
| 20  | 01ED92 | 000000011 | 110 | 110 | 110 | 010010 |  |
| 21  | 01ED94 | 000000011 | 110 | 110 | 110 | 010100 |  |
| 22  | 01A010 | 000000011 | 010 | 000 | 000 | 010000 |  |
| 23  | 018001 | 000000011 | 000 | 000 | 000 | 000001 |  |
| 24  | 062011 | 000001100 | 010 | 000 | 000 | 010001 |  |
| 25  | 010A16 | 000000010 | 000 | 101 | 000 | 010110 |  |
| 26  | 01A217 | 000000011 | 010 | 001 | 000 | 010111 |  |
| 27  | F59A01 | 111101011 | 001 | 101 | 000 | 000001 |  |
| 30  | 00A019 | 000000001 | 010 | 000 | 000 | 011001 |  |
| 31  | 010A01 | 000000010 | 000 | 101 | 000 | 000001 |  |

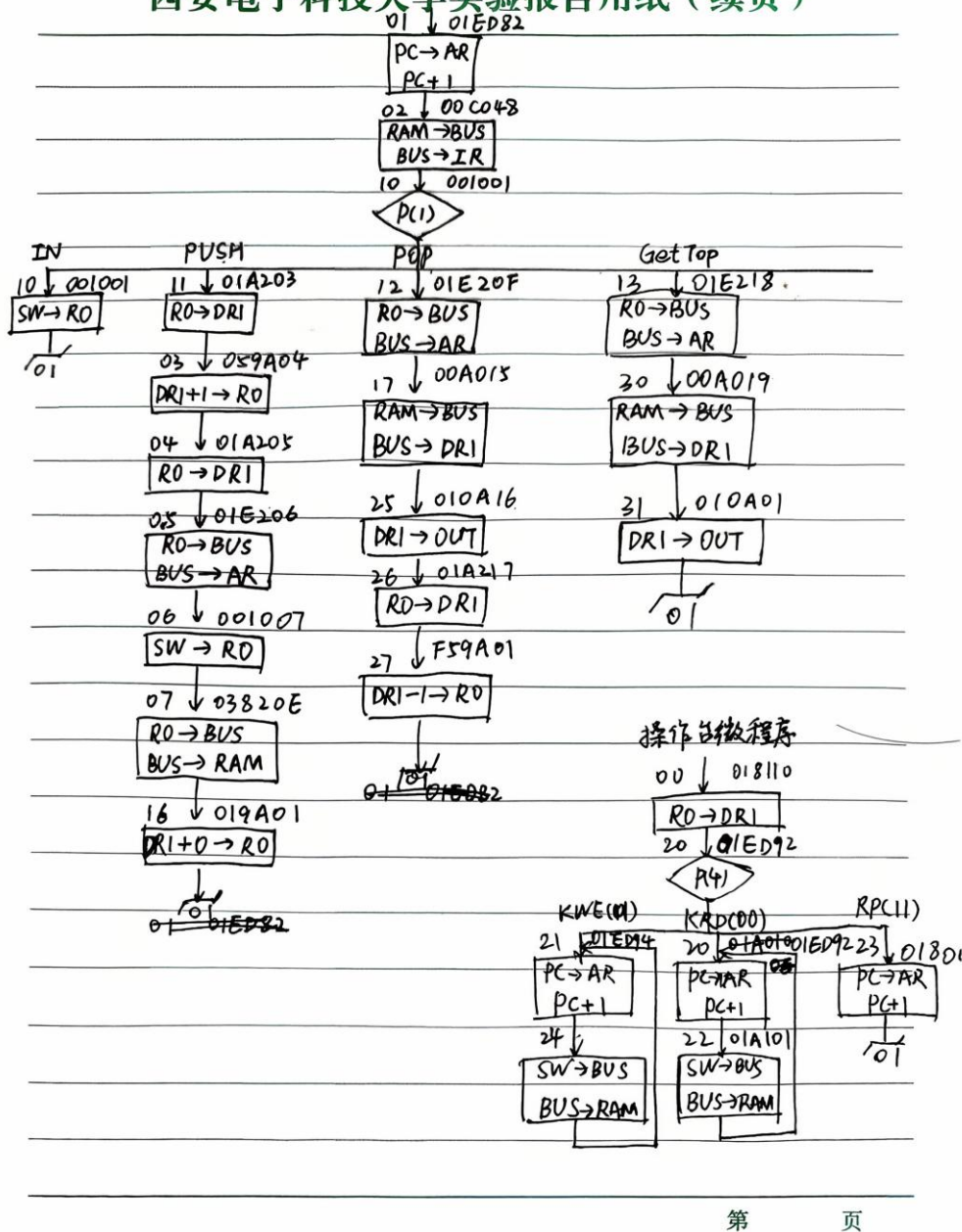
指令寄存器（IR）：指令寄存器用来保存当前正在执行的一条指令。当执行一条指令时，先把它从内存取到缓冲寄存器中，然后再传送至指令寄存器。指令划分为操作码和地址码段，由二进制数构成，为了执行任何给定的指令，必须对操作码进行测试“P(1)”，通过节拍脉冲T4的控制，以便识别所要求的操作。

指令译码器：根据指令中的操作码强置微控制器单元的微地址，使下一条微指令指向相应的微程序首地址。



实验程序：

# 西安电子科技大学实验报告用纸（续页）



微程序流程图

整体思路:

设计 RO 为栈指针，入栈时，先将 RO 通过 ALU 和 BUS 增加一位，将 RO 地址送入地址寄存器，控制台输入一个数字，送入 RAM 对应地址，再将 DRI 的地址复制给 RO。

出栈时，先将 RO 地址送入地址寄存器，输出对应地址的数据，然后 RO 减一。

三. 实验步骤

配置 LPM\_RAM

如果程序量大，手动输入效率太低，可以在计算机上编译好代码文件，并随同模型 CPU 设计文件一同编译进 SOF 下载文件中，直接下载进入 FPGA。

- (1) 在 QuartusII 环境下，打开工程文件 b100\_c.bdf，修改 b100\_C.bdf 中 LPM\_RAM\_DQ 的参数，将初始化文件 LPM\_FILE 设置为：“./5\_ram.mif”；打开“5\_ram.mif”（在示例中已有此文件），根据“实验程序 1”，在 5\_ram.mif 中输入全部机器指令代码（示例中已经输入）。
- (2) 将工程文件重新编译后，下载到实验台中，即完成 LPM\_RAM 的配置。
- (3) 根据以上的方法，复位信号 RST（键 8）=1；将控制开关 SWB、SWA（键 4、键 3）设置为：0、0，按键 7，每两个 2 次单步运行（产生 2 个正脉冲），检验配置进入 FPGA 中的程序代码。

（三）执行程序

实验中 LCD 液晶显示屏可以用来显示模型机 CPU 中各组成单元的内容。将 B100\_C.sof 文件下载到实验台后，按系统复位键，LCD 液晶显示屏即显示 CPU 中各组成单元的内容。其功能说明如下：



LCD 液晶显示屏功能说明

| 名称  | 作用          | 名称  | 作用       |
|-----|-------------|-----|----------|
| IN  | 输入单元 INPUT  | DR1 | 暂存器 DR1  |
| OUT | 输出单元 OUTPUT | DR2 | 暂存器 DR2  |
| ALU | 算术逻辑单元      | PC  | 程序计数器    |
| BUS | 内部数据总线      | AR  | 地址寄存器    |
| R0  | 寄存器 R0      | RAM | 程序/数据存储器 |
| R1  | 寄存器 R1      | IR  | 指令寄存器    |
| R2  | 寄存器 R2      | MC  | 微程序控制器   |



伪代码

| 助记符    | 指令码 | 微地址码 | 说明            |
|--------|-----|------|---------------|
| IN     | 00H | 10   | "INPUT"的数据→R0 |
| PUSH   | 10H | 11   | INPUT的数据入栈    |
| POP    | 20H | 12   | 出栈一个数据        |
| GetTop | 30H | 13   |               |

四. 实验结果分析

1.

| ROM_5.MIF |        |        |        |        |        |        |        |        |
|-----------|--------|--------|--------|--------|--------|--------|--------|--------|
| Addr      | +0     | +1     | +2     | +3     | +4     | +5     | +6     | +7     |
| 00        | 018110 | 01ED82 | 00C048 | 059A04 | 01A205 | 01E206 | 001007 | 03820E |
| 08        | 001001 | 01A203 | 01E20F | 01E218 | 000000 | 000000 | 019A01 | 00A015 |
| 10        | 01ED92 | 01ED94 | 01A010 | 018001 | 062011 | 010A16 | 01A217 | F59A01 |
| 18        | 00A019 | 010A01 | 000000 | 000000 | 000000 | 000000 | 000000 | 000000 |
| 20        | 000000 | 000000 | 000000 | 000000 | 000000 | 000000 | 000000 | 000000 |
| 28        | 000000 | 000000 | 000000 | 000000 | 000000 | 000000 | 000000 | 000000 |
| 30        | 000000 | 000000 | 000000 | 000000 | 000000 | 000000 | 000000 | 000000 |
| 38        | 000000 | 000000 | 000000 | 000000 | 000000 | 000000 | 000000 | 000000 |

ROM

| ROM_5.MIF |    |    |    |    |    |    |    |    |
|-----------|----|----|----|----|----|----|----|----|
| Addr      | +0 | +1 | +2 | +3 | +4 | +5 | +6 | +7 |
| 00        | 00 | 10 | 10 | 10 | 20 | 30 | 00 | 00 |
| 08        | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 10        | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 18        | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 20        | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 28        | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 30        | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 38        | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 40        | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 48        | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 50        | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 58        | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 60        | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 68        | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 70        | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 78        | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 80        | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 88        | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |

RAM

## 2.交换指令运行结果:



先输入 IN=10，第一次 10 入栈指令，R0=11，入栈 21，第二 10 入栈指令，R0=12 入栈 22，第三次 10 入栈指令，R0=13，入栈 23，20 出栈指令，OUT=23，30 输出栈顶元素 OUT=22.

## 五. 实验心得体会

通过本次计算机组成原理的课程设计实验，我学习了微指令的设计方法和工作原理。之前做计算机组成原理实验时也使用过一样的软件和实验箱，让我在这次的课程设计上对仪器操作的更加顺手。对我而言，这次的实验较难的地方在于指令的设计，再设计微指令上花费了我较多的时间去思考实现，设计时必须清楚地了解指令在计算机内部的存取地址及运行原理。指令设计好后在测试上并不是一次性通过的，通过自己的摸索与老师的细心讲解后，最终成功地将自己设计出的微指令。这次的课程设计也加深了我对微指令设计原理的理解。